

Chains in LangChain

p. 1

1 What is a chain?

A chain connects steps so the output of one step becomes the input of the next, automatically.

- Built with **LCEL** (LangChain Expression Language) using the pipe `|` operator.
- Every piece (prompt, model, parser, lambda) is a **Runnable**, so they all share the same methods: `.invoke()`, `.batch()`, `.stream()`.
- No manual passing of `result.content` between steps, and you can swap a model in one place.

2 Simple (sequential) chain



```
chain = prompt | model | parser
result = chain.invoke({'topic': 'Quantum Entanglement'})
```

3 Multi-step sequential chain

Feed one model's answer into a second prompt (report → 5-line summary):

```
template1 = PromptTemplate(template='Write a detailed report on {topic}',
                             input_variables=['topic'])
template2 = PromptTemplate(template='Write a 5 line summary on the following text. \n {text}',
                             input_variables=['text'])

chain = template1 | model | parser | template2 | model | parser
```

!! Rule: every prompt needs a **model** right after it.

`template1 | parser` → `ValidationError`, because the parser got a prompt, not model text.

→ the parser returns a plain **str**. `template2` has only one variable `{text}`, so a bare string is accepted and filled in.

4 See the chain

```
chain.get_graph().print_ascii()    # needs: pip install grandalf
```

Prints every step as boxes joined by lines. Great for checking the wiring.

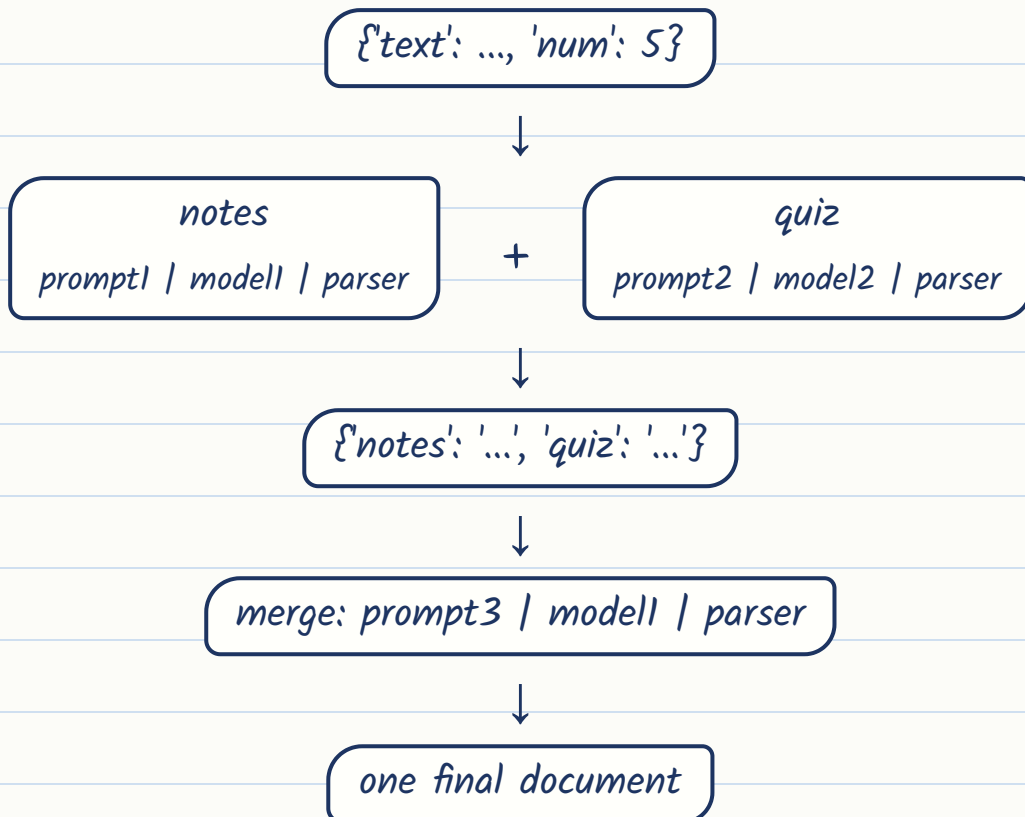
Parallel Chains

p. 2

1 The idea

`RunnableParallel` runs several chains at the same time on the same input.

Its output is a **dict**, with one key per branch.



2 Code

```
from langchain_core.runnables import RunnableParallel

parallel_chain = RunnableParallel({
    'notes': prompt1 | model1 | parser,
    'quiz': prompt2 | model2 | parser,
})

merge_chain = prompt3 | model1 | parser  # uses {notes} and {quiz}
final_chain = parallel_chain | merge_chain

result = final_chain.invoke({'text': text, 'num': 5})
```

3 Remember

- The dict **keys** (`notes`, `quiz`) must match the next prompt's variables `{notes}` and `{quiz}`.
- Every branch gets the whole input dict. A prompt just ignores keys it doesn't use (prompt1 ignores `num`).
- Branches can use different models (OpenAI + HuggingFace), but **if one branch fails, the whole chain fails**.
- Prompt tip: "Generate `{num}` short MCQs" works better than "short mcq type `{num}` questions".

4 See it

```
final_chain.get_graph().print_ascii()
```

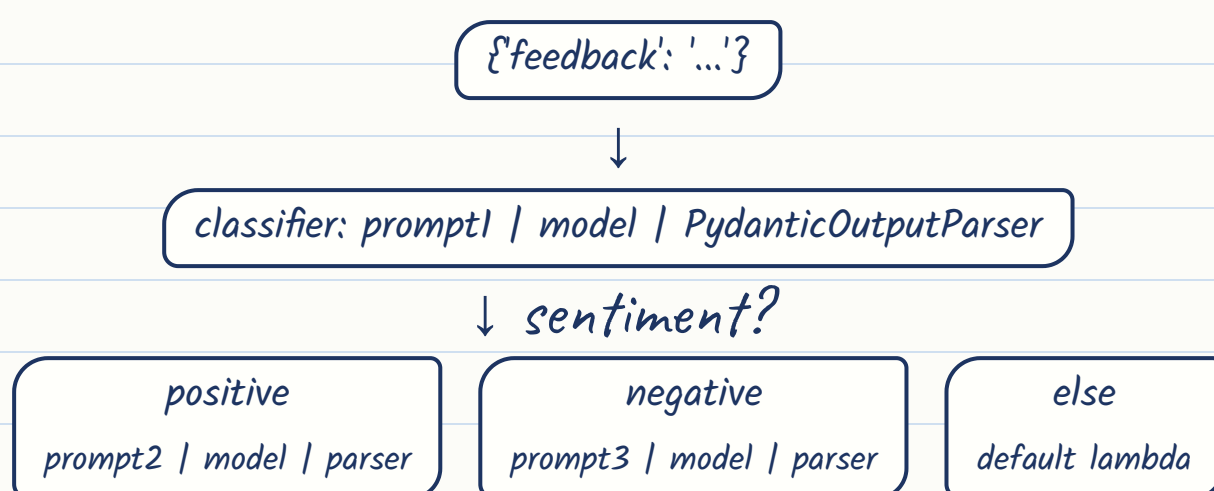
Shows the two branches side by side, joining into the merge prompt.

Conditional Chains

p. 3

Run a different chain depending on a condition, like if / elif / else.

Example: classify customer feedback, then reply differently for positive vs negative.



1 Classifier with a fixed set of labels

```
class Feedback(BaseModel):
    sentiment: Literal['positive', 'negative', 'other'] = Field(description='Sentiment')

parser2 = PydanticOutputParser(pydantic_object=Feedback)

prompt1 = PromptTemplate(
    template='Classify as positive, negative or other \n {feedback} \n {format_instruction}',
    input_variables=['feedback'],          # NOT format_instruction
    partial_variables={'format_instruction': parser2.get_format_instructions()}
)
classifier_chain = prompt1 | model | parser2
```

2 The branch

```
branch_chain = RunnableBranch(
    (lambda x: x['sentiment'].sentiment == 'positive', prompt2 | model | parser1),
    (lambda x: x['sentiment'].sentiment == 'negative', prompt3 | model | parser1),
    RunnableLambda(lambda x: 'Thanks for your feedback!') # default, no condition
)
```

3 Keep the original input!

```
chain = RunnablePassthrough.assign(sentiment=classifier_chain) | branch_chain
chain.invoke({'feedback': feedback})

# branch receives: {'feedback': '...', 'sentiment': Feedback(sentiment='positive')}
```

Without `.assign` the branch only gets `Feedback(...)`, so the reply prompt would answer `"sentiment='positive'"` instead of the real feedback.

4 Remember

- Each branch is a **tuple (condition, runnable)**: a comma, **not |**
- The default runnable goes **last**, with no condition.
- Pydantic validates **before** the branch: `'neutral'` not in the `Literal` → `OutputParserException`, and the branch never runs.
- List the allowed labels in the prompt too, so the model picks one of them.
- Safety net: `chain.with_fallbacks([RunnableLambda(lambda x: 'Could not classify')])`

Cheat Sheet

p. 4

1 Runnables

<code>RunnableSequence</code>	<code>a b c</code> , one after another
<code>RunnableParallel</code>	same input to all branches → dict output
<code>RunnableBranch</code>	(condition, chain) pairs + a default
<code>RunnableLambda</code>	turns any Python function into a step
<code>RunnablePassthrough</code>	passes input unchanged; <code>.assign()</code> adds keys
<code>.with_fallbacks([...])</code>	backup chain if any step raises

2 What each parser gives back

<code>StrOutputParser</code>	<code>str</code>
<code>JsonOutputParser</code>	dict (keys not enforced without a schema)
<code>StructuredOutputParser</code>	dict of strings, no type checks
<code>PydanticOutputParser</code>	validated <u>Pydantic object</u>

The chain's result is already the parsed value → never call `result.content` on it.

3 Imports

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser, PydanticOutputParser
from langchain_core.runnables import (RunnableParallel, RunnableBranch,
                                      RunnableLambda, RunnablePassthrough)

from langchain_classic.output_parsers import StructuredOutputParser, ResponseSchema # LangChain
l.x
```

4 Bugs I hit (don't repeat!)

1. `template | parser` with no model → `ValidationError`
2. `Responseschema`, `Runnablelambda` → names are **case-sensitive**
3. `result.content` after a parser → it's already a dict / str
4. `parser.parse()` on output that was already parsed in the chain
5. `invoke({'feedback'})` is a **set** → use `{'feedback': feedback}`
6. same variable in `input_variables` **and** `partial_variables` → overlap error
7. `lambda | chain` inside `RunnableBranch` → use a **comma**
8. model said 'neutral', `Literal` didn't allow it → `OutputParserException`
9. trailing space in API key → Illegal header value (`.strip()` it)
10. `ConnectTimeout` → network problem, not my code